

Research Sources & Context/Token Management Decisions

Multi-Agent Terminal Orchestration Project

This document consolidates every GitHub repository, tool, and primary source referenced while researching this project, plus a summary of the context/token management decisions made based on that research. It's meant as a bibliography/appendix alongside the BRD and Technical Documentation already produced.

1. GitHub Repositories & Tools Referenced

Core stack

Tool	URL	Role in this project
Wezterm	(terminal emulator, not GitHub-hosted in scope of this research)	Terminal rendering layer
HerdR	https://herdr.dev/	Pane/session orchestration — persistent PTYs, agent state tracking, CLI + JSON socket API

Context engineering / token optimization tools (the four originally supplied for evaluation)

Tool	URL	Verdict
Ponytail	https://github.com/DietrichGebert/ponytail	Adopted in reduced form — a condensed YAGNI system-prompt fragment, not the full plugin. Original benchmark numbers were contested and later revised by the author to ~54% less code, ~20%

Tool	URL	Verdict
		cheaper, ~27% faster on a real agentic benchmark (FastAPI+React repo)
rtk	https://github.com/rtk-ai/rtk	Adopted as default – CLI output compression proxy (git, tests, linters)
Code Context Engine (CCE)	https://github.com/rajkumarsakthivel/code-context-engine	Adopted, pilot-first – AST-aware shared retrieval index; flagged as early-stage/low-adoption at time of evaluation
AXI (Agent eXperience Interface)	https://axi.md/	Adopted as a design standard for any custom router/adaptor tooling built for this project

Additional tools found in the extended research pass

Tool	URL	Category	Verdict
TOON (Token-Oriented Object Notation)	https://github.com/toon-format/toon (spec: https://github.com/toon-format/spec)	Data serialization	Adopted – for plan/task-spec artifacts (uniform arrays of objects, TOON's ideal case); ~40–60% fewer tokens than JSON in published benchmarks
claude-presence	https://github.com/garniergeorges/claude-presence	Multi-session coordination	Adopted – presence registry, advisory resource locks, broadcast inbox for parallel Claude Code sessions on one repo; directly closes the concurrency/write

Tool	URL	Category	Verdict
			safety gap identified in review
ccsession	https://github.com/TimEvans/ccsession	Router/context-budget awareness	Pilot — exposes context-budget percentage, todo: and reset recommendation programmatically model for router headroom checks
context-mode	https://github.com/mksglu/context-mode	Combined tool-output compression + session memory	Pilot — reports up to 98% reduction on large tool output; potential single-tool consolidation of rtk + CCE, spans 17 platforms including Claude Code and Codex-adjacent tools
pi (coding agent / harness)	https://github.com/earendil-works/pi	Reference architecture	Referenced for its native subscription-logir pattern (<code>/login</code> Claude Pro/Max & ChatGPT Plus/Pro) and multi-provider model switching (<code>/model</code>) — informed the decision to wrap official <code>claude / codex</code> binaries directly rather than route through Pi
agent-comms	https://github.com/ExaDev/agent-comms	Peer-to-peer agent messaging	Adopted as the harness-agnostic P2P layer — TCP mesh on

Tool	URL	Category	Verdict
			localhost, bridges Claude Code, Codex, Pi, and OpenCode; used for lateral/exception-case clarification only, not default routing

Tools identified but not individually adopted (reference only)

Tool	Category	Note
headroom, lean-ctx, tokf, snip	rtk alternatives	Surfaced via GitHub's token-optimization topic listing; rtk remains the default given its maturity — these weren't individually deep-dived or URL-confirmed in this pass

2. Official Anthropic Platform Documentation & Research Referenced

Source	URL	What it informed
How we built our multi-agent research system	https://www.anthropic.com/engineering/multi-agent-research-system	The ~15x-tokens-vs-chat figure for Anthropic's own orchestrator-worker system the explicit caution that domains needing shared context or heavy inter-age dependency are a poor fit for this pattern; the "spawning 50 subagents f simple queries" failure mode that motivated the escalation/pre-check gate in this project

Source	URL	What it informed
When to use multi-agent systems (and when not to)	https://claude.com/blog/building-multi-agent-systems-when-and-how-to-use-them	The orchestrator-subagen pattern definition; context isolation guidance (most effective when a subtask generates high context volume but most of it is irrelevant to the main task the direct statement that teams have spent months on multi-agent architecture only to find single-agent prompting matched the result
Effective context engineering for AI agents	https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents	The "just-in-time" context strategy (lightweight identifiers/references loaded at runtime instead pre-loading everything) that underlies this project's Reference/Compressed/F tiering model; the distinction between prompt engineering and context engineering used throughout the technical doc
Claude Code subagents, compaction, context editing, memory tool, programmatic tool calling	Anthropic platform documentation (verify current specifics at https://docs.claude.com before implementation, since these evolve)	The native-mechanism baseline this project's harness is built to complement rather than duplicate — used throughout Section 12.1 of the technical doc

3. Context & Token Management Decisions Taken

A condensed record of the actual decisions, cross-referenced to where they live in the Technical Documentation:

1. **Reference > Compressed > Full, in that order of preference** for any content crossing a pane boundary — never default to sending raw/full content (Technical Doc §5).
2. **Structured data (TOON/JSON), not prose, for plans and task specs** — a verbose narrative plan gets re-paid for on every reference; a structured one can be sliced cheaply (§6.1, §12.2).
3. **Sonnet as an optional, measured middle tier between Opus and Haiku** — inserted only if Haiku's direct failure/retry rate on real chunks justifies the added coordination cost, never as a default (§6.1).
4. **Chain-of-command escalation by default; peer-to-peer (e.g., to Codex) only as the exception** — a blocked pane asks whoever assigned it the task first; lateral contact is reserved for cases where a peer specifically holds information the parent doesn't (§6.2).
5. **Conversational alignment (Phase 0) between the user and Opus before any plan is generated** — model tier selected via Claude Code's `/model` command; justified on token-cost grounds because unwinding a wrong plan downstream costs more than the alignment conversation itself (§6.1).
6. **Validation gate before compression, not after** — a result must pass a deterministic check (tests/lint/scope match) before it's allowed to be summarized; failures propagate in full detail rather than getting smoothed into a clean-sounding summary (§10.1).
7. **Git worktree per executing pane**, with merge-back explicitly owned by the planning pane — prevents concurrent panes from silently overwriting each other's file edits (§10.2).
8. **Explicit fallback chain for the Reference tier**: CCE index → plain file path → Compressed — so an immature dependency (CCE) degrading doesn't silently push the router toward the expensive end by default (§10.3).
9. **Deterministic (rule-based) tiering decisions by default**, with LLM-based judgment reserved for genuinely ambiguous handoffs and its cost counted explicitly — the router's own overhead is not allowed to hide inside claimed "savings" (§10.4).
10. **Concrete numeric acceptance criteria** rather than open-ended goals: $\geq 20\%$ token reduction vs. single-agent baseline, $\leq 5\%$ quality-score tolerance, tracked retry rates, and router overhead as its own measured line item (§11).
11. **Mandatory pre-build validation of concurrent CLI sessions** under one subscription login (rate-limit collisions, session-file contention) before any router code is written, with a queued/staggered fallback if it fails (§10.6, §11).
12. **A pre-check gate on the whole multi-pane approach**, not just the middle tier — grounded in the "baseline paradox" finding that coordination overhead produces negative returns once a single agent already handles a task class well (§6, citing §13).
13. **Native Anthropic context mechanisms (compaction, context editing, memory tool, programmatic tool calling) used within each pane**, with the custom harness

deliberately scoped to the cross-pane layer only — avoiding duplicated effort against what the platform already provides for free (§12.1).

14. **Official CLI binaries only, no third-party OAuth-token piggybacking** — because third-party harnesses drawing on subscription auth for programmatic use can be billed as extra usage rather than against plan limits, which would defeat the project's core cost objective.

4. Caveats on Sourcing

- Some tools (`headroom` , `lean-ctx` , `tokf` , `snip`) were surfaced via a GitHub topic listing but not individually deep-dived or link-verified in this pass — treat them as leads, not confirmed recommendations.
- Academic/benchmark figures referenced in the Technical Doc's Section 13 (the UIUC multi-agent token study, SWE-Pruner, the agent-harness-engineering survey) were characterized from search results during that research session but their exact URLs were not individually re-confirmed for this document — if you want to cite them formally, re-search and pull the direct paper links before relying on them externally.
- All URLs above were verified as live/correct as of this conversation (July 2026). GitHub star counts, tool maturity, and even repository ownership can shift quickly in this space — re-check before committing to a dependency, especially for the "pilot" and "reference only" tier tools.